

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Strings

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS



Recap on Strings

Recall from past lectures that:

- Strings are objects that can host letters, numbers, spaces, special characters.

Recap on Strings

Recall from past lectures that:

- Strings are objects that can host letters, numbers, spaces, special characters.

More in detail:

- strings are among the most popular types in Python
- we can create strings simply by enclosing characters in quotes
- Python treats single quotes as the same as double quotes (*Caveat: if your string contains a quote (e.g., `let 's dance`), enclose the string in double quotes. Similarly, if your string contains double quotes (e.g., `I say "hello"`), enclose the string in single quotes.*)
- to create a string, just perform a variable assignment

Examples:

```
var1 = 'Hello World!'
var2 = "Python is cool! (hopefully)"

print(var1)
print(var2)
```

```
Hello World!
Python is cool! (hopefully)
```

Length

To get the length of a string, we can use the `len()` function:

```
s = "Hello World"  
print("The string s is:", s)
```

The string s is: Hello World

```
n = len(s)  
print("The string is", n, "characters long")
```

The string is 11 characters long

No Python data type for single characters

Differently from other programming languages, Python does not have a data type for characters.

Hence, when dealing with single characters, Python will treat them as strings of length 1:

```
c = 'A'      # single character...  
print(type(c)) # ...still a string
```

```
<class 'str'>
```

Indexing

Using indexing, we can access a specific character of a string:

```
s = "Hello\nWorld!"  
print(s[0])      # we access the first character  
print(s[4])      # we access the fifth character  
print(s[len(s)-1]) # we access the last character  
  
i = 2  
print(s[i])      # we access the (i+1)-th character
```

H
o
!
l

WARNING: The first valid index is 0, not 1. The last valid index is `len(s) - 1`, not `len(s)`. This is called **zero-based indexing** or **0-indexing**.

Negative indexing

Quite strangely but conveniently, Python supports negative indices:

```
s = "Hello\nWorld!"  
print(s[-1])      # we access the last character  
print(s[-2])      # we access the second last character  
print(s[-len(s)]) # we access the first character
```

!
d
H

WARNING: The *first* negative index is `-1`, while the *last* is `-len(s)`.

Using an invalid index

When using an invalid index, Python will raise an error:

```
s = "Hello World"
print(s[20])
```

IndexError

Traceback (most recent call last)

Cell In[10], line 2

```
1 s = "Hello World"
----> 2 print(s[20])
```

IndexError: string index out of range

WARNING: Read the error from Python to fix your code issues!

Slicing

To access a substring (segment), i.e., a slice, use the **square brackets for slicing** along with the **index or indices** to obtain your substring. In particular, a **slice** can be gathered as:

```
string_name[start:stop]
```

NOTE: We include `start` in the slice, but we only go up to (and **not including**) `stop`. Further, indexing is **0-based** (the 1st element has index 0, the 2nd element has index 1 and so on ...).

Examples:

```
s = "Python is cool! (hopefully)"  
  
# slicing  
print(f's[1:5] = {s[1:5]}') # from second to fifth character  
  
s[1:5] = ytho
```

NOTE: The stop index is the first character that is **not** included in the slice.

The many shades of string slicing

Syntax of slicing:

- `s[start:stop]`: chars start through stop-1
- `s[start:]`: chars start through the rest of the string
- `s[:stop]`: chars from the beginning through stop-1
- `s[:]`: a full copy of the string

There is also the step value, which can be used with any of the above:

- `s[start:stop:step]`: start through not past stop, by step

start and stop can be negative numbers: this means that Python starts counting from the end of the string:

- `s[-1]`: last char in the string
- `s[-2:]`: last two chars in the string
- `s[:-2]`: everything except the last two chars

Examples:

```
s = 'hello'
print(f"s[1:4] -> {s[1:4]}")
print(f"s[1:] -> {s[1:]}")
print(f"s[:] -> {s[:]} (overestimating the upper bound)")
print(f"s[1:1000] -> {s[1:1000]}")
print(f"s[-1] -> {s[-1]} (reverse indexing)")
print(f"s[:] -> {s[:]}")
print(f"s[:-3] -> {s[:-3]} (everything except the last three chars)")
```

```
s[1:4] -> ell
s[1:] -> ello
s[:] -> hello (overestimating the upper bound)
s[1:1000] -> ello
s[-1] -> o (reverse indexing)
s[:] -> hello
s[:-3] -> he (everything except the last three chars)
```

Strings are immutable

A string is an immutable object, i.e., we cannot change its internal data. Hence, we cannot change an existing string:

```
s1 = "Hello\nWorld!"  
s1[5] = ", " # this will raise an exception
```

TypeError

Traceback (most recent call last)

Cell In[11], line 2

```
1 s1 = "Hello\nWorld!"  
----> 2 s1[5] = ", " # this will raise an exception
```

TypeError: 'str' object does not support item assignment

NOTE: *item assignment* means that you can't change a character in a string

Nonetheless, we can always create a new string:

```
s1 = "Hello\nWorld!"  
s2 = s1[:5] + ", " + s1[6:] # concatenation gives a new string  
print(s2)
```

Hello, World!

Operations on Strings

Recall from past lectures the two operators:

- **Concatenate:** with the + operator
- **Replicate:** with the * operator

Examples:

```
print(f'Hello " + "World!" -> {"Hello " + "World!"}')  
print(f'Python'*3 -> {"Python"*3})
```

```
"Hello " + "World!" -> Hello World!  
"Python"*3 -> PythonPythonPython
```


More operations on strings

Let `a = "Hello"` and `b = "Python"`.

Operator	Description	Example
<code>+</code>	Concatenation: adds values on either side of the operator	<code>a+b</code> yields <code>"HelloPython"</code>
<code>*</code>	Replication: creates new strings concatenating multiple copies of the original string	<code>a*2</code> yields <code>"HelloHello"</code>
<code>[]</code>	Slice: gives the character from the given index	<code>a[1]</code> yields <code>"e"</code>
<code>[:]</code>	Range: gives the character(s) from the given range	<code>a[1:4]</code> yields <code>"ell"</code>
<code>==</code>	Equality check: yields <code>True</code> if two strings are equal	<code>"Hello" == a</code> yields <code>True</code>
<code>!=</code>	Inequality check: yields <code>True</code> if two strings are not equal	<code>"Hello" != a</code> yields <code>False</code>
<code>in</code>	Membership: yields <code>True</code> if a substring appears in the string	<code>"He" in a</code> yields <code>True</code>
<code>not in</code>	Membership: yields <code>True</code> if a substring does not appear in the string	<code>"Me" not in a</code> yields <code>True</code>

Iterating over Strings

You can iterate over strings (i.e., get one character at the time) via `for` or `while` loops:

- Print each letter of `s` using a `for` loop:

```
s = "Python Program"
for letter in s:
    print(letter, end=' ')
```

P y t h o n P r o g r a m

- Print each letter of s using while loop:

```
s = "Python Program"
i = 0
while i < len(s):
    print(s[i], end=' ')
    i = i + 1
```

P y t h o n P r o g r a m

- Count the number of times 'a' appears in a string s using a for loop:

```
s = "Python Program"
count = 0
for letter in s:
    if letter == 'a':
        count = count + 1
print(f"Count of 'a': {count}")
```

Count of 'a': 1

- Count the number of times 'p' appears in s using a while loop:

```
s = "Python Program"
count = 0
i = 0
while i < len(s):
    if s[i] == 'p':
        count = count + 1
    i = i + 1
print(f"Count of 'p': {count} (because 'p' != 'P')")
```

Count of 'p': 0 (because 'p' != 'P')

Built-in String Methods

Python has a set of built-in methods that you can use on strings.

You can find the full list at <https://docs.python.org/3/library/stdtypes.html#string-methods>

WARNING: All string methods return new values. They do not change the original string.

`str.capitalize()`

`s.capitalize()` returns a copy of the string with its first character capitalised and the rest lowercased:

str.capitalize()

s.capitalize() returns a copy of the string with its first character capitalised and the rest lowercased:

```
s = "supercalifragilisticexpialidocious"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

str.capitalize()

s.capitalize() returns a copy of the string with its first character capitalised and the rest lowercased:

```
s = "supercalifragilisticexpialidocious"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

```
s = "SupercaliFragilisTicexpialid0ciouS"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

str.capitalize()

s.capitalize() returns a copy of the string with its first character capitalised and the rest lowercased:

```
s = "supercalifragilisticexpialidocious"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

```
s = "SupercaliFragilisTicexpialid0ciouS"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

```
# if the string is already capitalised, it has no effect  
s = "Supercalifragilisticexpialidocious"  
print(s.capitalize())
```

Supercalifragilisticexpialidocious

`str.lower()` and `str.upper()`

`s.lower()` returns a copy of the string with all the cased characters converted to lowercase.

`s.upper()` returns a copy of the string with all the cased characters converted to uppercase.

`str.lower()` and `str.upper()`

`s.lower()` returns a copy of the string with all the cased characters converted to lowercase.

`s.upper()` returns a copy of the string with all the cased characters converted to uppercase.

```
s = "SUPercalifragiliSTIcexpialidociouS"  
print(f'lower: {s.lower()}')  
print(f'upper: {s.upper()}')
```

lower: supercalifragilisticexpialidocious

upper: SUPERCALIFRAGILISTICEXPIALIDOCIOUS

`str.lower()` and `str.upper()`

`s.lower()` returns a copy of the string with all the cased characters converted to lowercase.

`s.upper()` returns a copy of the string with all the cased characters converted to uppercase.

```
s = "SUPercalifragiliSTIcexpialidocious"  
print(f'lower: {s.lower()}')  
print(f'upper: {s.upper()}')
```

```
lower: supercalifragilisticexpialidocious  
upper: SUPERCALIFRAGILISTICEXPIALIDOCIOUS
```

```
s = "supercalifragilisticexpialidocious"  
print(f'already lower: {s.lower()}')
```

```
already lower: supercalifragilisticexpialidocious
```

`str.lower()` and `str.upper()`

`s.lower()` returns a copy of the string with all the cased characters converted to lowercase.

`s.upper()` returns a copy of the string with all the cased characters converted to uppercase.

```
s = "SUPercalifragiliSTIcexpialidocious"  
print(f'lower: {s.lower()}')  
print(f'upper: {s.upper()}')
```

```
lower: supercalifragilisticexpialidocious  
upper: SUPERCALIFRAGILISTICEXPIALIDOCIOUS
```

```
s = "supercalifragilisticexpialidocious"  
print(f'already lower: {s.lower()}')
```

```
already lower: supercalifragilisticexpialidocious
```

```
s = "SUPERCALIFRAGILISTICEXPIALIDOCIOUS"  
print(f'already upper: {s.upper()}')
```

```
already upper: SUPERCALIFRAGILISTICEXPIALIDOCIOUS
```

`str.count()`

`s.count(sub, [start, end])` returns the number of non-overlapping occurrences of substring `sub` in the string `s`. Optionally, you may specify `start` and `end` to look for `sub` in a slice of `s`.

str.count()

`s.count(sub, [start, end])` returns the number of non-overlapping occurrences of substring `sub` in the string `s`. Optionally, you may specify `start` and `end` to look for `sub` in a slice of `s`.

```
s = "supercalifragilisticexpialidocious"  
print(f"s.count('li'): {s.count('li')}")
```

```
s.count('li'): 3
```


str.count()

`s.count(sub, [start, end])` returns the number of non-overlapping occurrences of substring `sub` in the string `s`. Optionally, you may specify `start` and `end` to look for `sub` in a slice of `s`.

```
s = "supercalifragilisticexpialidocious"  
print(f"s.count('li'): {s.count('li')}")
```

```
s.count('li'): 3
```

```
print(f"s.count('LI'): {s.count('LI')}")
```

```
s.count('LI'): 3
```

str.count()

`s.count(sub, [start, end])` returns the number of non-overlapping occurrences of substring `sub` in the string `s`. Optionally, you may specify `start` and `end` to look for `sub` in a slice of `s`.

```
s = "supercalifragilisticexpialidocious"  
print(f"s.count('li'): {s.count('li')}")
```

```
s.count('li'): 3
```

```
print(f"s.count('LI'): {s.count('LI')}")
```

```
s.count('LI'): 3
```

```
print(f"s.count('li', 0, 10): {s.count('li', 0, 10)}")
```

```
s.count('li', 0, 10): 1
```

str.count()

`s.count(sub, [start, end])` returns the number of non-overlapping occurrences of substring `sub` in the string `s`. Optionally, you may specify `start` and `end` to look for `sub` in a slice of `s`.

```
s = "supercalifragilisticexpialidocious"  
print(f"s.count('li'): {s.count('li')}")
```

```
s.count('li'): 3
```

```
print(f"s.count('LI'): {s.count('LI')}")
```

```
s.count('LI'): 3
```

```
print(f"s.count('li', 0, 10): {s.count('li', 0, 10)}")
```

```
s.count('li', 0, 10): 1
```

```
print(f"s[0:10].count('li'): {s[0:10].count('li')}")
```

```
s[0:10].count('li'): 1
```

`str.startswith()`

`s.startswith(sub, [start, end])` returns `True` if the string *starts* with the specified `sub`, otherwise returns `False`:

str.startswith()

`s.startswith(sub, [start, end])` returns `True` if the string *starts* with the specified `sub`, otherwise returns `False`:

```
s = "supercalifragilisticexpialidocious"  
print(f"s.startswith('super'): {s.startswith('super')}")
```

```
s.startswith('super'): True
```

str.startswith()

s.startswith(sub, [start, end]) returns True if the string *starts* with the specified sub, otherwise returns False:

```
s = "supercalifragilisticexpialidocious"
print(f"s.startswith('super'): {s.startswith('super')}")
```

```
s.startswith('super'): True
```

Use optional parameters start and end to test a slice of s:

```
print(f"s.startswith('super', 1, 10): {s.startswith('super', 1, 10)} (slice is 'upercalif')")
```

```
s.startswith('super', 1, 10): False (slice is 'upercalif')
```

str.startswith()

s.startswith(sub, [start, end]) returns True if the string *starts* with the specified sub, otherwise returns False:

```
s = "supercalifragilisticexpialidocious"  
print(f"s.startswith('super'): {s.startswith('super')}")
```

```
s.startswith('super'): True
```

Use optional parameters start and end to test a slice of s:

```
print(f"s.startswith('super', 1, 10): {s.startswith('super', 1, 10)} (slice is 'upercalif')")
```

```
s.startswith('super', 1, 10): False (slice is 'upercalif')
```

```
print(f"s.startswith('uper', 1, 10): {s.startswith('uper', 1, 10)}")
```

```
s.startswith('uper', 1, 10): True
```

`str.endswith()`

`s.endswith(sub, [start, end])` returns `True` if the string *ends* with the specified `sub`, otherwise returns `False`:

`str.endswith()`

`s.endswith(sub, [start, end])` returns `True` if the string *ends* with the specified `sub`, otherwise returns `False`:

```
print(f"s.endswith('ous'): {s.endswith('ous')}")
```

```
s.endswith('ous'): True
```

str.endswith()

s.endswith(sub, [start, end]) returns True if the string *ends* with the specified sub, otherwise returns False:

```
print(f"s.endswith('ous'): {s.endswith('ous')}")
```

```
s.endswith('ous'): True
```

```
print(f"s.endswith('calif', 1, 10): {s.endswith('calif', 1, 10)}")
```

```
s.endswith('calif', 1, 10): True
```

`str.find()` and `str.index()`

`s.find(sub, [start, end])` returns the lowest index in the string `s` where substring `sub`. If `sub` is not found, it returns `-1`:

`str.find()` and `str.index()`

`s.find(sub, [start, end])` returns the lowest index in the string `s` where substring `sub`. If `sub` is not found, it returns `-1`:

```
s = "supercalifragilisticexpialidocious"  
print(f"s.find('calif'): {s.find('calif')}")
```

```
s.find('calif'): 5
```

`str.find()` and `str.index()`

`s.find(sub, [start, end])` returns the lowest index in the string `s` where substring `sub`. If `sub` is not found, it returns `-1`:

```
s = "supercalifragilisticexpialidocious"  
print(f"s.find('calif'): {s.find('calif')}")
```

```
s.find('calif'): 5
```

`s.index(sub, [start, end])` works like `s.find()`, but raises `ValueError` when the substring is not found:

```
print(f"s.index('calif'): {s.index('calif')}")
```

```
s.index('calif'): 5
```

`str.find()` and `str.index()`

`s.find(sub, [start, end])` returns the lowest index in the string `s` where substring `sub`. If `sub` is not found, it returns `-1`:

```
s = "supercalifragilisticexpialidocious"  
print(f"s.find('calif'): {s.find('calif')}")
```

```
s.find('calif'): 5
```

`s.index(sub, [start, end])` works like `s.find()`, but raises `ValueError` when the substring is not found:

```
print(f"s.index('calif'): {s.index('calif')}")
```

```
s.index('calif'): 5
```

```
print(f"s.find('calio'): {s.find('calio')}")
```

```
s.find('calio'): -1
```

```
s.index('calio')
```

ValueError

Traceback (most recent call last)

Cell In[18], line 1

----> 1 s.index('calio')

ValueError: substring not found

HINT: Use `find()` or `index()` if you need the position of the substring. To check if sub exists in `s`, you may use the `in` operator..

```
print(f"'calif' in s: {'calif' in s}")
```

```
'calif' in s: True
```


`str.is*()`

- `s.islower()`: returns True if a non-empty string `s` contains only lowercase characters.
- `s.isupper()`: returns True if a non-empty string `s` contains only uppercase characters.
- `s.isalnum()`: returns True if a non-empty string `s` contains only alphanumeric characters (numbers and letters).
- `s.isalpha()`: returns True if a non-empty string `s` contains only alphabetic characters (letters).
- `s.isdecimal()`: returns True if a non-empty string `s` contains only decimal numbers.
- `s.isdigit()`: returns True if a non-empty string `s` contains only digits (decimals, subscripts and superscripts).
- `s.isnumeric()`: returns True if a non-empty string `s` contains only numeric characters (decimals, subscripts, superscripts, vulgar fractions, roman numerals, currency numerators).
- `s.istitle()`: returns True if a non-empty string `s` is titlecased.

